

Rapport de tests automatises et indicateurs de qualite

Plateforme MSPR HealthAI Coach (MSPR3 / TPRES601). Etat releve le 10/06/2026.

Ce document complete le plan de test (`plan-de-test.md`) : il rapporte les resultats reels des dernieres executions de tests en integration continue et les indicateurs de qualite de code mesures. La strategie de test (types de tests, declencheurs, outils par service) est decrite dans le plan de test.

1. Derniers runs CI par service

Chaque depot execute sa chaine GitHub Actions (lint, tests, build, publication GHCR). Etat des derniers runs sur la branche par default :

Service	Branche	Dernier run	Resultat	Lien
API	<code>main</code>	08/06/2026	Succes	run 27145460311
Auth	<code>main</code>	08/06/2026	Succes	run 27143017850
AI-Nutrition	<code>master</code>	09/06/2026	Succes	run 27188678630
Reco-Fitness	<code>master</code>	08/06/2026	Succes	run 27144710950
ETL	<code>master</code>	20/04/2026	Succes	run 24667234255
Front	<code>main</code>	10/06/2026	Succes	run 27270853990
BDD	<code>main</code>	26/05/2026	Succes	run 26444280468
MongoDB	<code>master</code>	22/05/2026	Succes	run 26278826349

Les 8 chaines sont au vert. La publication d'image est conditionnee aux tests : un job en echec bloque la publication sur GHCR (verifie en pratique, voir 1.1).

Capture de la file de runs de l'API : `captures/actions-api-runs.png`.

1.1 Incident e2e Front (resolu le 10/06/2026)

La suite e2e Cypress du Front etait en echec depuis le 26/05/2026 (2 specs sur 6), bloquant la publication de l'image, conformement au comportement attendu du pipeline. Causes identifiees et corrigees :

- `meal-analysis.cy.ts` : mock manquant de `GET /api/v1/me/macros`, appele au montage de la vue ; sans lui la vue affiche l'ecran "profil incomplet" a la place de la dropzone.
- `meal-plan.cy.ts` : fixture `meal-plan.json` restee sur une ancienne structure (`breakfast / lunch / dinner` par jour) au lieu du contrat actuel `days[].meals[]` (champ `est_budget_eur` inclus).

Apres correction, la suite complete passe : 6 specs, 10 tests, 0 echec (verifie en local puis en CI, run 27270853990).

2. Couverture de tests

Service	Outil	Resultat mesure	Seuil	Enforcement
API	JaCoCo	>= 80 % (lignes et branches)	80 %	Bloquant : regle <code>jacoco-check</code> a la phase <code>verify</code> , le build vert garantit le seuil
AI-Nutrition	pytest-cov	92 % (run du 09/06/2026)	cible 80 %	Mesure en CI, rapports HTML/XML publies en artefacts
Reco-Fitness	pytest-cov	85 % (run du 08/06/2026)	80 %	Mesure en CI sur le rapport <code>term-missing</code>
Auth	bun test	suites passantes	-	Tests sans seuil
ETL	pytest	unitaires + integration PostgreSQL passants	-	Tests sans seuil
Front	Vitest + Cypress	unitaires + 6 specs e2e passants	-	Tests sans seuil

Les services sans code applicatif (BDD, MongoDB) sont valides par execution réelle (migrations SQL sequentielles, init des collections et index) a chaque run.

3. Indicateurs SonarCloud

L'analyse statique est assuree par SonarCloud (application GitHub de l'organisation `whitefoxyt`, mode Automatic Analysis : analyse a chaque push, sans token dans les depots). Mesures relevees le 10/06/2026 via l'API publique :

Projet	Lignes de code	Bugs	Vulnerabilites	Hotspots securite	Code smells	Duplication	Derniere analyse
AI-Nutrition	11 863	30	3	18	102	0,7 %	09/06/2026
Reco-Fitness	6 120	35	1	8	19	1,5 %	08/06/2026
MongoDB	importe, aucune analyse declenchee a ce jour						

Notes associees (echelle A a E) : maintenabilite A sur les deux projets, fiabilite C, securite E (tiree par les vulnerabilites ouvertes, 3 et 1 respectivement). Ces points constituent le backlog qualite a traiter en continu.

Limites connues :

- La couverture de tests n'apparait pas dans SonarCloud : l'Automatic Analysis n'execute pas les tests. La couverture fait foi en CI (section 2).
- Les depots API, Auth, ETL, Front et BDD ne sont pas encore importes dans l'organisation SonarCloud ; l'import se fait depuis l'interface SonarCloud par un administrateur de l'organisation, sans configuration cote

depot. L'API conserve par ailleurs un environnement SonarQube local (`docker-compose.sonar.yml`) pour une analyse manuelle.

Captures : `captures/sonarcloud-ai-nutrition.png` , `captures/sonarcloud-reco-fitness.png` .

4. Verification d'exploitation (smoke test du 10/06/2026)

Demarrage complet de la stack en configuration complete (application + supervision) sur un poste de developpement, images GHCR et modele LLM deja telecharges :

- `docker compose up -d` (base + monitoring) : stack stable en **84 secondes**, pull des 9 images de supervision inclus (l'exigence du cahier des charges est < 10 minutes).
- 20 conteneurs actifs, healthchecks au vert (le front local etait indisponible lors du releve : port 5173 deja occupe par un autre processus sur le poste, sans lien avec la plateforme).
- Prometheus : 10 cibles sur 10 a l'etat UP (4 services applicatifs instrumentes, exporters infrastructure et bases de donnees).
- Les 3 regles d'alerte chargees, a l'etat inactif (aucune condition d'alerte remplie).
- Loki : logs centralises recus de 23 conteneurs.
- Grafana : 4 tableaux de bord provisionnes dans le dossier "MSPR HealthAI" (Vue stack, Observabilite logs + metriques, API Spring Boot, Microservices FastAPI).

Captures des tableaux de bord en fonctionnement dans `../02-observabilite-supervision/captures/` (`dashboard-vue-stack.png` , `dashboard-observabilite.png` , `dashboard-api-spring.png`).